BÀI GIẢNG CHI TIẾT & ĐẦY ĐỦ: BÀI TOÁN N-QUEENS

DỰ ÁN NGHIÊN CỨU & SO SÁNH 3 PHƯƠNG PHÁP AI/TỐI ƯU HÓA

Chủ đề: Mã hóa SAT (5 Thuật toán AMO), Constraint Programming (CP) và Integer Linear Programming (ILP)

Đối tượng: Giảng dạy từ con số 0 dành cho sinh viên / thành viên nhóm chưa có nền tảng.
Nội dung: Lý thuyết cốt lõi, Ví dụ trực quan bàn cờ, Công thức toán, Mã nguồn & Phân tích Đồ thị Benchmark.

Sinh viên: Bùi Đình Cảnh MSSV: 24021392 Lớp: INT3011E_3
Email: buidinhcanh2006@gmail.com

Bộ giải sử dụng: PySAT/Glucose3, OR-Tools CP-SAT, Gurobi MIP, CPLEX MIP và CPLEX CP Optimizer
Môi trường thực nghiệm: Google Colab (2 CPU logic; timeout 15/60/120 giây tùy phase; các lượt Gurobi bổ sung ở phiên khác)

SLIDE 1: GIỚI THIỆU BÀI TOÁN N-QUEENS

1. Khái niệm Bài toán:

- Đặt N quân Hậu lên bàn cờ kích thước N × N.
- Quy tắc quân Hậu: Có thể di chuyển ngang, dọc và chéo không giới hạn ô.
- Mục tiêu: Không có 2 quân Hậu nào khống chế lẫn nhau.

2. Ba ràng buộc cốt lõi:

1. Ràng buộc Hàng: Mỗi hàng có đúng 1 quân Hậu.
2. Ràng buộc Cột: Mỗi cột có đúng 1 quân Hậu.
3. Ràng buộc Đường chéo: Tối đa 1 quân Hậu trên mỗi đường chéo chính và phụ.

Minh họa Bàn cờ 4 × 4:

```
BÀN CỜ 4x4 (N=4) VỚI LỜI GIẢI HỢP LỆ:
  +---+---+---+---+
0 | . | Q | . | . |   -> Hậu tại ô (0, 1)
  +---+---+---+---+
1 | . | . | . | Q |   -> Hậu tại ô (1, 3)
  +---+---+---+---+
2 | Q | . | . | . |   -> Hậu tại ô (2, 0)
  +---+---+---+---+
3 | . | . | Q | . |   -> Hậu tại ô (3, 2)
  +---+---+---+---+
    0   1   2   3
```

Kiểm tra: Không có 2 quân Hậu nào nằm chung Hàng, Cột hoặc Đường chéo!

SLIDE 2: BỨC TRANH TỔNG QUAN 3 PHƯƠNG PHÁP GIẢI

Tiêu chí	1. SAT (Glucose3)	2. CP (OR-Tools)	3. ILP (Gurobi)
Biểu diễn Biến	Biến Mệnh đề Nhị phân $x_{r,c}$ nhận True hoặc False	Biến Vị trí Cột Nguyên Q_i nhận 0 .. N-1	Ma trận Biến 0-1 $x_{r,c}$ nhận 0 hoặc 1
Dạng Ràng buộc	Các mệnh đề CNF (AND của các OR): ALO & AMO	Ràng buộc toàn cục (Global Constraint): AllDifferent	Hệ Phương trình & Bất phương trình Đại số Tuyến tính
Ưu điểm Cốt lõi	Thuật toán CDCL cực mạnh cho bài toán logic quy mô	Mã nguồn ngắn gọn (chỉ 3 dòng toán), lan truyền ràng buộc tốt	Thuật toán Simplex & Branch-and-Bound tối ưu toán học
Thử thách	Bùng nổ số lượng mệnh đề nếu chọn mã hóa kém	Bùng nổ không gian tìm kiếm khi N lớn	Bản quyền thương mại giới hạn kích thước biến

Quy trình xử lý chung: Bài toán N-Queens -> Chuyển đổi mô hình (Encoding) -> Bộ giải (Solver) -> Trích xuất nghiệm bàn cờ.

SLIDE 3: CỐT LÕI CỦA SAT SOLVER VÀ CHUẨN CNF

1. Dạng chuẩn CNF (Conjunctive Normal Form):

- Mọi bài toán SAT được phát biểu dưới dạng Tích của các Tổng (AND của các OR).
- Cấu trúc: C_1 AND C_2 AND ... AND C_m (mỗi C_i là một clause / mệnh đề).
- Mỗi ô (r, c) trên bàn cờ tương ứng 1 biến nhị phân: $x_{r,c}$ = 1 (True) nếu có Hậu, ngược lại 0 (False).

2. Hai loại Ràng buộc Logic bắt buộc:

A. ALO (At-Least-One - Tối thiểu 1): Ít nhất 1 ô trong hàng có Hậu.

=> Mệnh đề CNF cho hàng r: ($x_{r,0}$ OR $x_{r,1}$ OR ... OR $x_{r,N-1}$)

B. AMO (At-Most-One - Tối đa 1): Nhiều nhất 1 ô có Hậu trên mỗi hàng/cột/đường chéo.

=> Đảm bảo không có 2 quân Hậu nào khống chế nhau. Có 5 cách mã hóa AMO khác nhau!

SLIDE 4: CHI TIẾT 5 THUẬT TOÁN MÃ HÓA AT-MOST-ONE (AMO)

Thuật toán AMO	Cách hoạt động & Ý tưởng	Độ phức tạp	Đánh giá Thực nghiệm
1. Pairwise (Cặp đôi)	Xét mọi cặp ô (i, j). Mệnh đề: (NOT x_i OR NOT x_j).	Biến phụ: 0 Mệnh đề: $O(N^2)$	Số clause tăng nhanh; không chạy ở phase large để tránh chi phí dự kiến quá lớn.
2. Sequential (Nối tiếp)	Dùng chuỗi biến phụ s_i để 'đếm tích lũy' xem từ ô 1 đến i đã có Hậu xuất hiện chưa.	Biến phụ: O(N) Mệnh đề: O(N)	Nhanh nhất ở N=16, chậm rõ rệt tại N=64 và timeout từ N=128 trong bộ dữ liệu này.
3. Commander (Chỉ huy)	Chia N ô thành các nhóm nhỏ. Mỗi nhóm tạo 1 'Biến chỉ huy' c_g đại diện.	Biến phụ: O(N) Mệnh đề: O(N)	Hoàn thành đến N=512; ba run N=1000 nhận SIGKILL, chưa đủ log để kết luận OOM.
4. Product (Tích 2D)	Mã hóa vị trí ô bằng giao điểm hàng phụ r_i và cột phụ c_j trong lưới 2D (p x q).	Biến phụ: O(sqrt(N)) Mệnh đề: O(sqrt(N))	Timeout ở N=32,64 nhưng hoàn thành cả ba run ở N=1000.
5. Binary (Nhị phân)	Dùng ceil($\log_2$ N) bit nhị phân để mã hóa chỉ số vị trí ô chứa quân Hậu.	Biến phụ: O(log N) Mệnh đề: O(N log N)	Hoàn thành N=128, 256, 512 nhưng timeout tại N=64 và N=1000.

SLIDE 5: LẬP TRÌNH RÀNG BUỘC (CP VỚI OR-TOOLS)

1. Tư duy Biểu diễn Biến trong CP:

- Thay vì dùng N x N biến nhị phân như SAT, CP sử dụng N biến số nguyên.
- Đặt Q_i nhận giá trị 0, 1, ..., N-1 là vị trí cột của quân Hậu ở hàng i.
- Miền giá trị (Domain): Mỗi biến Q_i có N giá trị có thể nhận.

2. Mã nguồn 3 dòng Toán gọn nhẹ trong Python:

```
from ortools.sat.python import cp_model
model = cp_model.CpModel()
Q = [model.NewIntVar(0, n - 1, f'Q_{i}') for i in range(n)]

# Ràng buộc 1: Không trùng Cột
model.AddAllDifferent(Q)

# Ràng buộc 2: Không trùng Đường chéo chính
model.AddAllDifferent([Q[i] + i for i in range(n)])

# Ràng buộc 3: Không trùng Đường chéo phụ
model.AddAllDifferent([Q[i] - i for i in range(n)])
```

Đánh giá: Mô hình rất gọn. OR-Tools hoàn thành 2/3 run tại N=256 với timeout 120 giây, rồi timeout ở N=512 và N=1000.

SLIDE 6: QUY HOẠCH TUYẾN TÍNH NGUYÊN (ILP VỚI GUROBI)

1. Biểu diễn Bài toán thành Hệ Bất Phương Trình Đại Số:

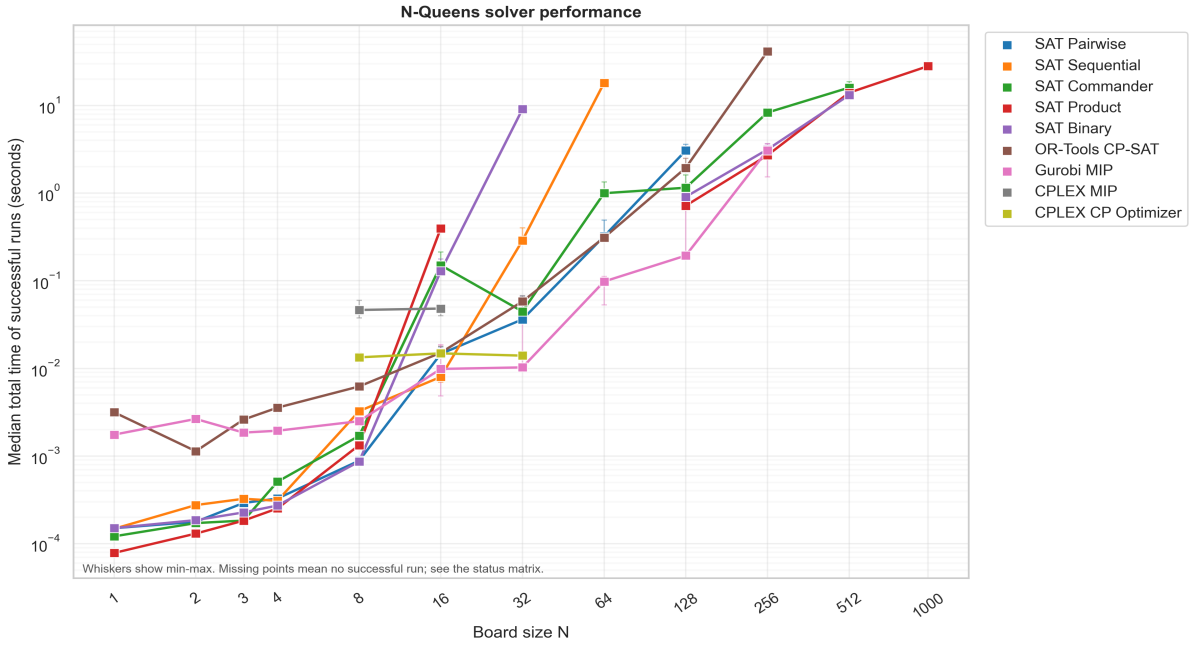
- Sử dụng ma trận biến nhị phân $x_{r,c}$ nhận 0 hoặc 1.
- Tổng theo hàng = 1: sum($x_{r,c}$, c=0..N-1) = 1 với mọi r.
- Tổng theo cột = 1: sum($x_{r,c}$, r=0..N-1) = 1 với mọi c.
- Đường chéo chính <= 1: sum($x_{r,c}$, r-c=k) <= 1 với mọi k.
- Đường chéo phụ <= 1: sum($x_{r,c}$, r+c=k) <= 1 với mọi k.

2. Gurobi Academic WLS:

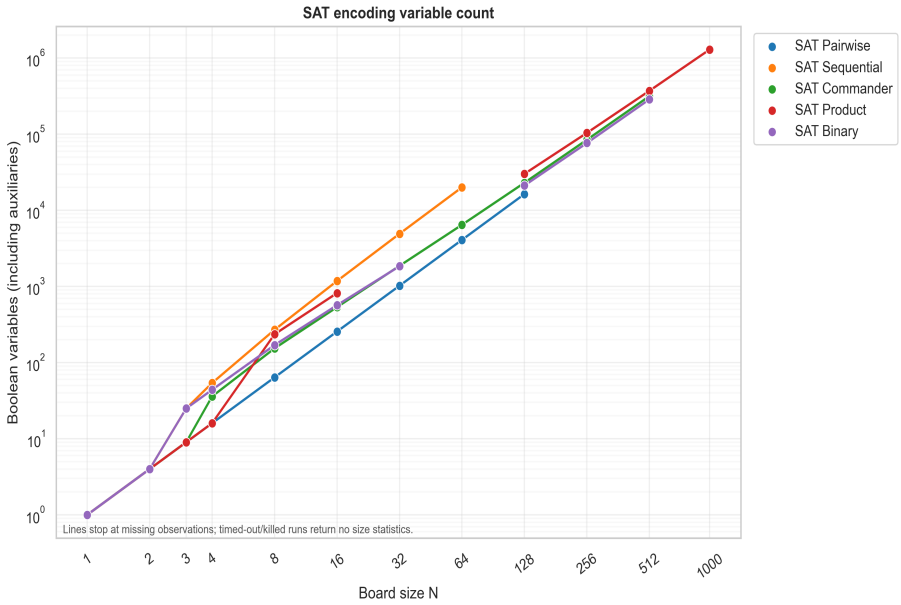
- Gurobi là bộ giải ILP thương mại hàng đầu thế giới.
- Academic WLS cho phép chạy lại các cấu hình từng bị giới hạn: N=64,128 đều SAT 5/5; N=256 SAT 3/3. Trung vị tương ứng 0.098, 0.194 và 3.078 giây.

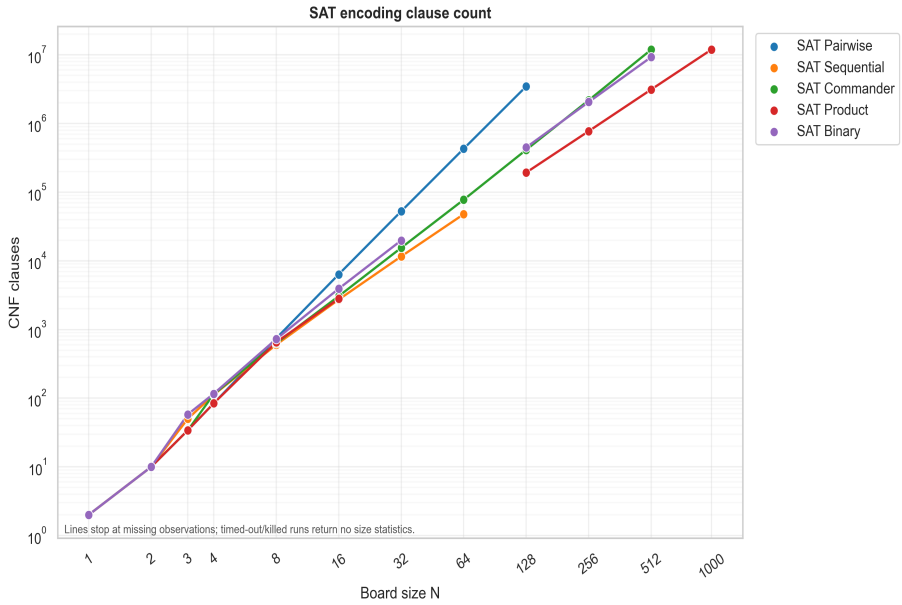
SLIDE 7: KẾT QUẢ BENCHMARK THỜI GIAN CHẠY (SEC)

N	Pairwise	Sequential	Commander	Product	Binary	CP (OR-Tools)	ILP (Gurobi)
4	0.000s	0.000s	0.001s	0.000s	0.000s	0.004s	0.002s
8	0.001s	0.003s	0.002s	0.001s	0.001s	0.006s	0.003s
16	0.015s	0.008s	0.151s	0.397s	0.129s	0.015s	0.010s
32	0.036s	0.288s	0.045s	TIMEOUT	9.090s	0.058s	0.010s
64	0.322s	18.143s	0.998s	TIMEOUT	TIMEOUT	0.311s	0.098s
128	3.063s	TIMEOUT	1.153s	0.719s	0.908s	1.944s	0.194s
256	—	TIMEOUT	8.309s	2.704s	3.147s	41.473s (2/3)	3.078s
512	—	TIMEOUT	16.003s	14.004s	13.158s	TIMEOUT	—
1000	—	TIMEOUT	KILLED	28.127s	TIMEOUT	TIMEOUT	—



SLIDE 8: PHÂN TÍCH SỐ LƯỢNG BIẾN VÀ MỆNH ĐỀ SAT





Nhận xét đồ thị: Pairwise không dùng biến phụ nhưng số clause tăng nhanh. Product dùng nhiều biến phụ hơn, song là encoding duy nhất hoàn thành cả ba run tại N=1000. Run timeout/killed không trả về quy mô encoding nên đường biểu diễn dừng tại điểm thiếu.

SLIDE 9: GIẢI THÍCH 3 HIỆN TƯỢNG KỸ THUẬT ĐẶC BIỆT

1. Giới hạn giấy phép còn lại thuộc CPLEX:

• Ba lượt CPLEX MIP tại N=32 có trạng thái LICENSE_LIMIT. Academic WLS của Gurobi không cấp phép cho CPLEX; toàn bộ 13 lượt Gurobi đo lại đều SAT hợp lệ.

2. Commander bị hệ điều hành dừng tại N=1000:

• Ba run có exit code -9 (SIGKILL). Không có log kernel nên chỉ gắn nhãn KILLED, không khẳng định nguyên nhân là hết RAM.

3. Hiện tượng 'Đuôi Nặng' (Heavy-Tailed Phenomenon) của Product SAT:

• Product timeout ở N=32,64 nhưng hoàn thành 3/3 run tại N=1000 với trung vị 28.127 giây.

• Diễn giải thận trọng: thời gian CDCL phụ thuộc cấu trúc encoding và heuristic, nên độ khó không nhất thiết tăng đơn điệu theo N. Dữ liệu hiện có không đủ để khẳng định một cơ chế nhân quả cụ thể.

SLIDE 10: TỔNG KẾT VÀ CÂU HỎI PHẢN BIỆN HỘI ĐỒNG

1. Tổng kết Phương pháp Tốt nhất:

- Không có phương pháp thắng ở mọi N: Sequential nhanh nhất tại N=16, Gurobi tại N=32/64/128, Product tại N=256/1000 và Binary tại N=512. Các lượt Gurobi mới chạy trong phiên Colab khác.
- Khi báo cáo: luôn trình bày thời gian cùng tỷ lệ hoàn thành, timeout, license limit và SIGKILL; không suy rộng ngoài môi trường đã đo.

2. Bộ Câu hỏi Phản biện Hội đồng Thường gặp:

Q1: Em tự viết Solver hay dùng thư viện?

=> Trả lời: Em xây dựng pipeline CNF và tích hợp đủ 5 encoding trong sat_encoder.py. Pairwise, Sequential và Binary dùng CardEnc của PySAT; Commander và Product được cài đặt trong project; Glucose3 giải CNF thu được.

Q2: Tại sao CP lại viết code ngắn hơn SAT rất nhiều?

=> Trả lời: Vì CP hỗ trợ ràng buộc toàn cục AllDifferent, bộ giải tự lan truyền ràng buộc mức biến nguyên thay vì phải phân rã thành mệnh đề nhị phân như SAT.